LAB ASSIGNMNET 11.3

Subject : AI Assistant Coding

Hall Ticket No : 2303A52465

Name : M.Avinash

Batch : 50

Date : 11/3/26

Task-1 :

Prompt : Analyze the given scenario and Implement the contact manager using arrays (lists) and implement the same functionality using a linked list for dynamic memory allocation. Implement the following operations in both approaches:

- Add a contact
- Search for a contact
- Delete a contact

```python
# SR University's student club requires a simple Contact Manager Application
# to store members' names and phone numbers. The system should support
# efficient addition, searching, and deletion of contacts.
# Analyze the given scenario and Implement the contact manager using arrays (lists) and implement the same functionality using a linked list for dynamic memory allocation.
# Implement the following operations in both approaches:
#    Add a contact
#    Search for a contact
#    Delete a contact
# Contact Manager using Arrays (Lists)
class ContactManagerArray:
    def __init__(self):
        self.contacts = []

    def add_contact(self, name, phone):
        self.contacts.append({'name': name, 'phone': phone})

    def search_contact(self, name):
        for contact in self.contacts:
            if contact['name'] == name:
                return contact
        return None

    def delete_contact(self, name):
        for i, contact in enumerate(self.contacts):
            if contact['name'] == name:
                del self.contacts[i]
                return True
        return False
# Contact Manager using Linked List
class ContactNode:
    def __init__(self, name, phone):
        self.name = name
        self.phone = phone
        self.next = None

class ContactManagerLinkedList:
    def __init__(self):
        self.head = None

    def add_contact(self, name, phone):
        new_node = ContactNode(name, phone)
        new_node.next = self.head
        self.head = new_node

    def search_contact(self, name):
        current = self.head
        while current:
            if current.name == name:
                return {'name': current.name, 'phone': current.phone}
            current = current.next
        return None
```

```python
    def delete_contact(self, name):
        current = self.head
        prev = None
        while current:
            if current.name == name:
                if prev:
                    prev.next = current.next
                else:
                    self.head = current.next
                return True
            prev = current
            current = current.next
        return False
# Example usage
if __name__ == "__main__":
    # Using Contact Manager with Arrays
    array_manager = ContactManagerArray()
    array_manager.add_contact("Alice", "1234567890")
    array_manager.add_contact("Bob", "0987654321")
    print(array_manager.search_contact("Alice"))  # {'name': 'Alice', 'phone': '1234567890'}
    print(array_manager.delete_contact("Bob"))  # True
    print(array_manager.search_contact("Bob"))  # None

    # Using Contact Manager with Linked List
    linked_list_manager = ContactManagerLinkedList()
    linked_list_manager.add_contact("Charlie", "5555555555")
    linked_list_manager.add_contact("Dave", "4444444444")
    print(linked_list_manager.search_contact("Charlie"))  # {'name': 'Charlie', 'phone': '5555555555'}
    print(linked_list_manager.delete_contact("Dave"))  # True
    print(linked_list_manager.search_contact("Dave"))  # None
```

Output :

```
{'name': 'Alice', 'phone': '1234567890'}
True
None
{'name': 'Charlie', 'phone': '5555555555'}
True
None
PS C:\Users\GOPAL\OneDrive\Desktop\AI AC\Lab-11>
```

Task-2 :

Prompt : Analyze the given scenario and Implement a Queue (FIFO) to manage book requests, Extend the system to a Priority Queue, prioritizing faculty requests.
Use AI to assist in generating enqueue() method and dequeue() method
Test the system with a mix of student and faculty requests.

```python
# The SRU Library manages book borrow requests. Students and faculty submit
# requests, but faculty requests must be prioritized over student requests.
# Analyze the given scenario and Implement a Queue (FIFO) to manage book requests, Extend the system to a Priority Queue, prioritizing faculty requests.
# Use AI to assist in generating enqueue() method and dequeue() method
# Test the system with a mix of student and faculty requests.
class BookRequest:
    def __init__(self, requester_type, requester_name, book_title):
        self.requester_type = requester_type  # 'student' or 'faculty'
        self.requester_name = requester_name
        self.book_title = book_title
class BookRequestQueue:
    def __init__(self):
        self.queue = []

    def enqueue(self, request):
        self.queue.append(request)

    def dequeue(self):
        if not self.queue:
            return None
        return self.queue.pop(0)
class PriorityBookRequestQueue:
    def __init__(self):
        self.faculty_queue = []
        self.student_queue = []

    def enqueue(self, request):
        if request.requester_type == 'faculty':
            self.faculty_queue.append(request)
        else:
            self.student_queue.append(request)

    def dequeue(self):
        if self.faculty_queue:
            return self.faculty_queue.pop(0)
        elif self.student_queue:
            return self.student_queue.pop(0)
        else:
            return None
# Example usage
if __name__ == "__main__":
    # Using BookRequestQueue
    book_queue = BookRequestQueue()
    book_queue.enqueue(BookRequest('student', 'Alice', 'Book A'))
    book_queue.enqueue(BookRequest('faculty', 'Dr. Smith', 'Book B'))
    print(book_queue.dequeue().requester_name)  # Alice
    print(book_queue.dequeue().requester_name)  # Dr. Smith

    # Using PriorityBookRequestQueue
    priority_queue = PriorityBookRequestQueue()
    priority_queue.enqueue(BookRequest('student', 'Bob', 'Book C'))
    priority_queue.enqueue(BookRequest('faculty', 'Dr. Johnson', 'Book D'))
    priority_queue.enqueue(BookRequest('student', 'Charlie', 'Book E'))
    print(priority_queue.dequeue().requester_name)  # Dr. Johnson
    print(priority_queue.dequeue().requester_name)  # Bob
    print(priority_queue.dequeue().requester_name)  # Charlie
```

Output :

```
PS C:\Users\GOPAL\OneDrive\Desktop\AI AC\Lab-11> & C:/Python314/python.exe "c:/Users/GOPAL/OneDrive/Desktop/AI AC/Lab-11/Lab-11.py"
Alice
Dr. Smith
Dr. Johnson
Bob
Charlie
PS C:\Users\GOPAL\OneDrive\Desktop\AI AC\Lab-11>
```

Task-3 :

Prompt : Analyze the given scenario and mplement a Stack to manage support tickets. Provide the following operations:

- push(ticket)
- pop()
- peek()

Simulate at least five tickets being raised and resolved, Use AI to suggest additional stack operations such as: Checking whether the stack is empty, Checking whether the stack is full (if applicable)

```python
# SR University's IT Help Desk receives technical support tickets from students
# and staff. While tickets are received sequentially, issue escalation follows a
# Last-In, First-Out (LIFO) approach.
# Analyze the given scenario and mplement a Stack to manage support tickets.
# Provide the following operations:
#   push(ticket)
#   pop()
#   peek()
# Simulate at least five tickets being raised and resolved, Use AI to suggest additional stack operations such as: Checking whether the stack is empty, Checking whether the stack is full (if applicable)
class SupportTicket:
    def __init__(self, ticket_id, requester_name, issue_description):
        self.ticket_id = ticket_id
        self.requester_name = requester_name
        self.issue_description = issue_description
class SupportTicketStack:
    def __init__(self):
        self.stack = []

    def push(self, ticket):
        self.stack.append(ticket)

    def pop(self):
        if not self.is_empty():
            return self.stack.pop()
        return None

    def peek(self):
        if not self.is_empty():
            return self.stack[-1]
        return None

    def is_empty(self):
        return len(self.stack) == 0
    def is_full(self, max_size):
        return len(self.stack) >= max_size
# Example usage
if __name__ == "__main__":
    ticket_stack = SupportTicketStack()
    ticket_stack.push(SupportTicket(1, 'Alice', 'Cannot connect to Wi-Fi'))
    ticket_stack.push(SupportTicket(2, 'Bob', 'Computer is slow'))
    ticket_stack.push(SupportTicket(3, 'Charlie', 'Printer not working'))
    ticket_stack.push(SupportTicket(4, 'Dave', 'Software installation issue'))
    ticket_stack.push(SupportTicket(5, 'Eve', 'Email not syncing'))

    print(ticket_stack.peek().requester_name)  # Eve
    print(ticket_stack.pop().requester_name)  # Eve
    print(ticket_stack.pop().requester_name)  # Dave
    print(ticket_stack.is_empty())  # False
    print(ticket_stack.is_full(5))  # False
    ticket_stack.push(SupportTicket(6, 'Frank', 'Keyboard not working'))
    print(ticket_stack.is_full(5))  # True
```

Output :

```
PS C:\Users\GOPAL\OneDrive\Desktop\AI AC\Lab-11> & C:/Python314/python.exe "c:/Users/GOPAL/OneDrive/Desktop/AI AC/Lab-11/Lab-11.py"
Eve
Eve
Dave
False
False
False
PS C:\Users\GOPAL\OneDrive\Desktop\AI AC\Lab-11>
```

Task-4 :

Prompt : Use AI and Generate a code To implement a Hash Table and understand collision handling and generate a hash table with: Insert, Search, Delete according to the given Starter Code

Expected Outcome

- Collision handling using chaining
- Well-commented methods

```python
# Starter Code
# class HashTable:
# pass
# Use AI and Generate a code To implement a Hash Table and understand collision handling and generate a hash table with: Insert, Search, Delete according to the given Starter Code
# Expected Outcome
#  Collision handling using chaining
#  Well-commented methods

class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]  # Create a list of empty lists for chaining

    def hash_function(self, key):
        """Generate a hash for the given key."""
        return hash(key) % self.size

    def insert(self, key, value):
        """Insert a key-value pair into the hash table."""
        index = self.hash_function(key)
        # Check if the key already exists and update it
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                self.table[index][i] = (key, value)  # Update existing key
                return
        # If the key does not exist, add a new key-value pair
        self.table[index].append((key, value))

    def search(self, key):
        """Search for a value by its key."""
        index = self.hash_function(key)
        for k, v in self.table[index]:
            if k == key:
                return v  # Return the value if found
        return None  # Return None if the key is not found

    def delete(self, key):
        """Delete a key-value pair from the hash table."""
        index = self.hash_function(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                del self.table[index][i]  # Remove the key-value pair
                return True  # Return True if deletion was successful
        return False  # Return False if the key was not found
# Example usage
if __name__ == "__main__":
    hash_table = HashTable()
    hash_table.insert("name", "Alice")
    hash_table.insert("age", 30)
    hash_table.insert("city", "New York")

    print(hash_table.search("name"))  # Output: Alice
    print(hash_table.search("age"))   # Output: 30
    print(hash_table.search("city"))  # Output: New York

    hash_table.delete("age")
    print(hash_table.search("age"))   # Output: None
```

Output :

```
PS C:\Users\GOPAL\OneDrive\Desktop\AI AC\Lab-11> & C:/Python314/python.exe "c:/Users/GOPAL/OneDrive/Desktop/AI AC/Lab-11/Lab-11.py"
Alice
30
New York
None
PS C:\Users\GOPAL\OneDrive\Desktop\AI AC\Lab-11>
```

Task-5 :

Prompt : Generate a code to Design a Campus Resource Management System with the following features:

- Student Attendance Tracking
- Event Registration System
- Library Book Borrowing
- Bus Scheduling System
- Cafeteria Order Queue

Student Tasks :

- Choose the most appropriate data structure for each feature.
- Justify your choice in 2–3 sentences.
- Implement one selected feature using AI-assisted code generation.

```python
class CampusResourceManagementSystem:
    def __init__(self):
        self.attendance = {}  # Using a dictionary for O(1) access to student attendance
        self.event_registrations = []  # Using a list to store event registrations in order
        self.library_books = {}  # Using a dictionary for O(1) access to book availability
        self.bus_schedule = []  # Using a list to maintain the bus schedule in order
        self.cafeteria_queue = []  # Using a list to manage the cafeteria order queue in FIFO manner

    def track_attendance(self, student_id, present):
        """Track student attendance."""
        self.attendance[student_id] = present

    def register_event(self, student_id, event_name):
        """Register a student for an event."""
        self.event_registrations.append((student_id, event_name))

    def borrow_book(self, student_id, book_title):
        """Allow a student to borrow a book if available."""
        if book_title in self.library_books and self.library_books[book_title] > 0:
            self.library_books[book_title] -= 1
            return True
        return False

    def schedule_bus(self, bus_number, departure_time):
        """Schedule a bus departure."""
        self.bus_schedule.append((bus_number, departure_time))

    def place_cafeteria_order(self, student_id, order_details):
        """Place an order in the cafeteria queue."""
        self.cafeteria_queue.append((student_id, order_details))
# Example usage
if __name__ == "__main__":
    campus_system = CampusResourceManagementSystem()
    campus_system.track_attendance("S001", True)
    campus_system.register_event("S001", "Tech Talk")
    campus_system.library_books["Data Structures"] = 5
    print(campus_system.borrow_book("S001", "Data Structures"))  # Output: True
    campus_system.schedule_bus("Bus 1", "08:00 AM")
    campus_system.place_cafeteria_order("S001", "Veg Sandwich")
    campus_system.place_cafeteria_order("S002", "Chicken Wrap")
    print(campus_system.cafeteria_queue)  # Output: [('S001', 'Veg Sandwich'), ('S002', 'Chicken Wrap')]
    campus_system.place_cafeteria_order("S003", "Pasta")
    print(campus_system.cafeteria_queue)  # Output: [('S001', 'Veg Sandwich'), ('S002', 'Chicken Wrap'), ('S003', 'Pasta')]
    print(campus_system.borrow_book("S002", "Data Structures"))  # Output: False
```

Output :

```
True
[('S001', 'Veg Sandwich'), ('S002', 'Chicken Wrap')]
[('S001', 'Veg Sandwich'), ('S002', 'Chicken Wrap'), ('S003', 'Pasta')]
True
PS C:\Users\GOPAL\OneDrive\Desktop\AI AC\Lab-11>
```